

The best optimization was the one I decided *not to ship*.

I built an agent that reads a small online store's support inbox and writes reply drafts. I thought the hard part would be the writing.

It was the cost.

THE SYSTEM

This ran on a real inbox, not in a playground.

453

emails processed
in production

0

sent without
human review

66%

escalated to
a person

It runs every two minutes. For each email it picks one of three actions: draft, escalate, discard. Everything that can be settled in code is settled before the model sees the case: proving identity, adding up deadlines, filtering noise.

The model writes. A person sends. Always.

An 89% cache hit rate looked solved.

The log stored no token counts at all. The only source was the monthly invoice, aggregated and impossible to attribute to a single email. Before optimizing anything, I instrumented it.

\$0.0107

reading the prefix
that is already cached

\$0.2147

rewriting it on
a cold start

Writing costs twenty times what reading costs. Which is why *the 11% of misses cost 2.4× more than every hit combined.*

The problem was never that the prompt was large. It was how often it got rewritten.

There were two cache entries, not one.

The output schema is part of the cached prefix. Because the two calls used different schemas, they never shared anything. On a cold start, one escalation rewrote the entire prompt twice.

The hypothesis was obvious: one schema, one entry, and half the writes disappear.

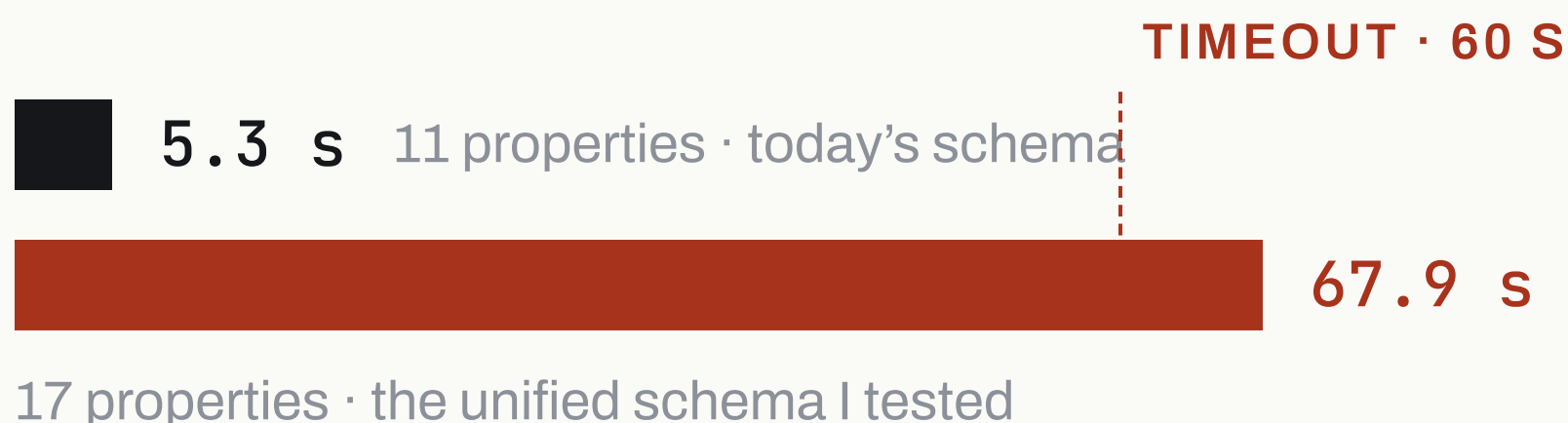
I could have shipped it that afternoon. Everyone would have agreed it was an improvement.

I TESTED IT FIRST

MEASURED · THE TEST COST \$0.008

It would have failed on every call.

Response time, by number of properties in the output schema



This is not degradation, it is a stop. Past the timeout it fails every time, not some of the time. And the cost is not linear, so trimming a field or two would not have saved it either.

An optimization rejected after a test *is a good engineering outcome.*

66% escalated. I wanted that number down.

The easy read is “there is a lot left to automate here”. I spent a day reading the reasons, one by one.

Action on an order	103	needs write access
Earlier commitment	95	state no system records
Human judgment	52	warranty, dispute, exception
Six other categories	47	context, identity, gap

I tested the most promising hypothesis, the refunds: **2 of 9** were already recorded. The other 7 really were unprocessed.

Getting that number down meant giving the agent write access to the store, *which is exactly the guardrail that makes this tolerable.*

A prompt guardrail can be talked around. An infrastructure one cannot.

No Mail.Send

The application never requested send permission from Microsoft Graph. It is not a rule in the code that someone can work around. It is a permission that does not exist.

Read-only store

The agent reads orders and writes none. Nothing it decides can move money or change a shipment.

Even if every other one fails at the same time, the worst possible outcome is *a wrong draft that a person reads and deletes.*

No email goes out. No order changes.

Every edit the owner makes is an unwritten requirement.

When he changes a draft before sending it, he is correcting a rule I never wrote.

-
- 01 Compare what was generated with what was actually sent
 - 02 Group by pattern, not by email
 - 03 At 7pm the system sends him three, on Discord
 - 04 He answers. A person writes the rule
-

Ask *why*, never propose the answer. A polite yes gives you a worse rule than no rule at all.

In the 3 September round: one question produced no rule, one widened a trigger, and one exposed a defect of mine.

Cost fell 63%. It was not free.

	AUG 30–31	SEP 2–3	
Paid tokens per email	19,998	7,443	-63%
Model calls per email	1.73	1.00	-42%
Emails resolved with a draft	23%	40%	+17 pts
Drafts sent without a word changed	79%	74%	-5 pts

The last row is the price. The system started drafting cases it used to escalate, and taking on harder cases pushes down the share that goes out untouched.

Five points for seventeen looks like a good trade to me. But it is a trade, and I would rather write it down than round it off.

This does not prove causation: three changes landed in the same window.

The right metric was never how many cases I automated. *It was how many I could automate safely.*

An average can hide an asymmetry in price, and you only see it after instrumenting. A hypothesis rejected by measurement is a result. And not all the automation left undone is debt.

What is the optimization you measured and decided not to ship?